

Static Impact Analysis Does Not Transfer: A Pre-Registered Refutation of Two LLM-Agent Reliability Mechanisms

Anonymous Author(s)

Abstract

Coding agents built on frozen language models are widely argued to need external mechanisms that supply what the model structurally lacks: a memory of the codebase, a prediction of what an edit will break, and a gate that refuses to act on an under-specified request. We built two such mechanisms and reported strong results on self-constructed evaluations: a dependency-graph *impact oracle* with perfect recall (1.000) of impacted files, and a complexity *router* with an assumption *gate* achieving $F1 = 1.00$ and a 62.1% cost saving. This paper reports what happened when we evaluated both against data we did not construct, under a protocol frozen before any data was collected.

Both results collapse. On 759 files across nine independently selected open-source Python repositories, with co-change ground truth mined from real commit history, the oracle’s pooled recall is 2.20% (95% CI [1.20, 3.31]) — not 100% — and 91.0% of files receive *exactly zero* recall. A `grep` baseline with no dependency graph at all attains recall 0.573 and $F1$ 0.437 against the oracle’s 0.042. On 80 held-out tasks drawn from real GitHub issues, with every threshold left untouched, gate $F1$ falls from 1.00 to 0.37, and the cost saving does not merely shrink but inverts: counting the pipeline’s actual total spend, routing costs 20.2% *more* than always using the premium tier.

We report three things beyond the refutation. First, a structural decomposition of all 18,221 false negatives showing that 96.8% are *not* a ground-truth ceiling but an artifact of reverse-only graph traversal, together with a construction defect whose safe repair alone recovers an 11.0× recall gap. Second, a repair of both defects evaluated under the same frozen protocol with a declared tuning/held-out repository split. Third, the observation that our central composition law is decades-old layer-of-protection algebra, and was independently and more generally derived weeks before our own write-up. We argue the transferable contribution is the protocol, not the mechanisms.

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; *Software evolution*.

Keywords

LLM coding agents, change impact analysis, co-change, negative results, pre-registration, verification gates

1 Introduction

A developer opening an unfamiliar file carries context the file does not contain. They remember which callers depend on it, they anticipate what a signature change will break, and when a request is too vague to act on they ask instead of guessing. A frozen language

model has none of this: it is a stateless map from a bounded context window to a token sequence, and each invocation begins with no memory of the last.

This observation has motivated a large and growing class of *external* mechanisms that supply the missing faculties around an unmodified model — persistent stores, dependency-graph analyses, verification hooks, and routing layers. The argument for them is structural and, we still believe, correct. What is far less clear is whether any particular mechanism delivers what its authors measure.

We are those authors. We built two mechanisms and evaluated them the way such work is commonly evaluated: on material we constructed ourselves. An *impact oracle* parsed a Python codebase into a dependency graph and answered “what will this edit break” by reverse reachability with per-hop confidence decay; on a ten-file package with five injected mutations it recovered every impacted file, recall 1.000. A complexity *router* classified tasks into cost tiers and an assumption *gate* halted under-specified requests; on thirty hand-written tasks it achieved $F1 = 1.00$ and a 62.1% cost saving against an always-premium baseline.

Both evaluations were honestly labelled as demonstrations. Neither survived contact with data we did not build.

This paper reports that failure and what we learned from anatomising it. We pre-registered a protocol — corpus eligibility criteria, commit filters, thresholds, and bootstrap seeds — and froze it before cloning a single repository, with the explicit stipulation that an unfavourable number would be reported rather than reconciled. We then evaluated the oracle against co-change ground truth mined from nine independently selected open-source projects, and the router and gate against tasks drawn from real GitHub issues, with every threshold left exactly as shipped.

Contributions.

- (1) A **pre-registered evaluation protocol** for LLM-agent impact mechanisms, using repository co-change history as ground truth, with all thresholds and filters frozen before data collection and an audited append-only amendment trail (§2).
- (2) A **refutation** of both mechanisms’ headline results on real data: oracle recall $1.000 \rightarrow 0.022$, gate $F1$ $1.00 \rightarrow 0.37$, and a cost saving that inverts to -20.2% once escalation retries are counted (§3, §5).
- (3) A **structural failure-mode taxonomy** attributing 96.8% of missed impact to a traversal-direction blind spot rather than to a limit of the ground truth, and isolating a graph-construction defect whose safe repair recovers an 11.0× recall gap (§4, §6).

- (4) A **repair and re-evaluation** of both defects under the same frozen protocol, with a declared tuning/held-out repository split (§6).
- (5) An **honest novelty accounting**: our composition law is an instance of layer-of-protection analysis, and was independently and more generally derived weeks before our own write-up (§8).

What we do not claim. We do not claim that external impact analysis is worthless for coding agents, that co-change is a substitute for semantic impact, or that our repaired mechanism is production-ready. We claim that the evidence standard commonly used to justify such mechanisms — a self-built demonstration — is capable of overstating recall by more than an order of magnitude, and we demonstrate this on our own work.

2 Protocol

The evaluation was pre-registered. Before any repository was cloned or any commit mined, we fixed and froze: nine numbered corpus eligibility criteria; the commit filters defining ground truth; the confidence-threshold grid; the bootstrap procedure and its seed; and the stipulation that governs everything else — *if a downstream number looks bad, the fix is to report it, not to edit the protocol*. Amendments were permitted only as dated, append-only addenda. Three were filed, all documenting the corpus-selection funnel; a character-level diff confirms nothing above the freeze line was altered.

Corpus. Candidates required Python purity ≥ 0.85 , ≥ 3000 stars, ≥ 50 contributors, ≥ 800 commits, age ≥ 5 years, activity within 24 months, repository size under 500 MB, no affiliation with this project, and ≥ 15 Python files after path exclusions. The grep baseline extends the prototype's own symbol-search routine to file granularity; matching is on identifier occurrences, so short or generic identifiers can inflate its false-positive count, which if anything favours the oracle in the comparison. Twelve primary candidates were tried in a pre-declared order with substitution from a pre-declared backup list; substitutions were permitted only in response to these metadata criteria, never in response to performance. Nine repositories qualified: requests, flask, click, jinja, rich, httpx, attrs, pytest, and loguru. Each was pinned to an exact commit SHA recorded in the corpus artifact.

Ground truth. For a file f , the ground-truth impact set is the set of other Python files appearing in the same non-merge commit as f , aggregated over history. Merges, mega-commits above the pre-declared size threshold (formatting sweeps, vendored dependency bumps, mass renames), and excluded paths were dropped by the frozen filters. This yields 801 labelled files, of which 759 met the evaluation's inclusion condition.

Ground truth is a proxy, and its ceiling is measurable. Co-change is not semantic impact. Files co-change for reasons no static analysis can see (release chores, coordinated documentation, test-fixture coupling), and files that *should* co-change sometimes do not. We therefore measured the ceiling directly: the fraction of ground-truth pairs connected by *any* static path in either direction, at any hop, in the graph the oracle itself builds. Pooled, 96.9% of ground-truth impact is statically reachable in principle. That number bounds every

Table 1: Pooled, micro-averaged over 759 files in 9 repositories, with 95% bootstrap CIs ($B = 2000$, file-resampled). The oracle is shown at $t = 0.02$, the most favourable point of the pre-registered grid for its recall.

Method	Precision	Recall	F1
Impact oracle ($t=0.02$)	0.398	0.022	0.042
Impact oracle (shipped $t=0.10$)	0.381	0.017	0.033
Grep baseline (no graph)	0.353	0.573	0.437
Edited-file-only	0.000	0.000	0.000
<i>Static ceiling on this ground truth</i>	—	<i>0.969</i>	—

result below, and it is what makes the observed recall interpretable rather than merely low. Two caveats: it is measured on the graph the *as-shipped* oracle builds, so repairing the construction defect (§6) raises it slightly, to roughly 97.2%; and reachability in a dense graph is a weak property, so the ceiling is an upper bound on what any static method could attain, not evidence that a reachable pair is causally related.

Independent reproduction. Before computing any metric, we mined all nine repositories' histories from scratch and compared against the supplied ground truth: 801 of 801 rows matched exactly. Two harness corrections were made at this stage — both identified and disclosed *before* any metric was computed.

3 Evaluation I: Impact Closure

Table 1 states the result. The oracle's pooled recall is 0.0220, 95% CI [0.012, 0.033]. The claim under test was 1.000. The oracle reaches 2.27% of its own static ceiling.

The per-file distribution shows this is not a uniform shortfall but close to a step function: 691 of 759 files (91.0%) receive *exactly* zero recall — the oracle misses every co-changed file — and one file achieves recall 1.0. A mean of 2.2% conceals a system that, for nine files in ten, returns nothing useful at all.

The grep baseline is the uncomfortable comparison. A textual search for symbol names, with no dependency graph, no confidence model, and no persistent state, attains recall 0.573 and F1 0.437 against the oracle's 0.042. Whatever a codebase world-model is supposed to buy over an unstructured baseline, it is not visible here. Precision is the one metric where the oracle is competitive (0.398 versus 0.353): when it warns, it is often right. It simply almost never warns.

4 Failure Modes

A negative result is only useful if it explains itself. We classified all 18221 pooled false negatives structurally, by testing reachability in the oracle's own graph.

The decomposition in Table 2 is the paper's most actionable finding. Only 3.19% of missed impact is attributable to the proxy nature of co-change. The remaining 96.8% is a path the oracle's own graph already contains but its traversal cannot reach: 94.7% are *sibling* pairs, where A and B both import a common module C and co-change when C 's contract shifts, yet neither is reverse-reachable from the other. The mechanism was not defeated by the messiness

Oracle blast-radius prediction vs. real co-change history, 9 OSS repos (micro-avg, threshold=0.02, 95% bootstrap CI)

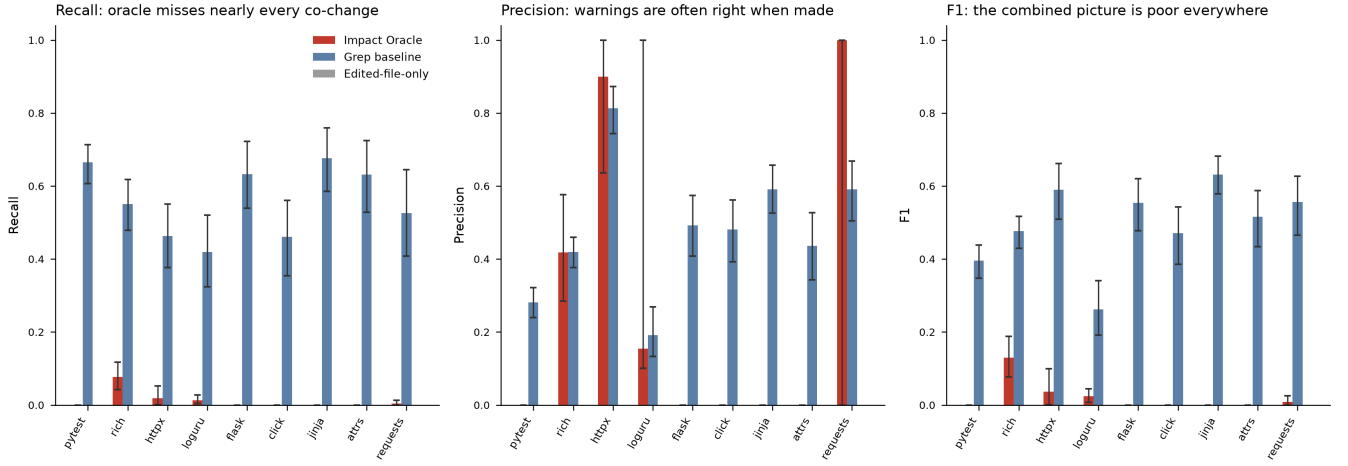


Figure 1: Per-repository recall, precision, and F1 with 95% bootstrap CIs. The oracle’s recall bars are near-invisible against the grep baseline in every repository.

Table 2: Structural decomposition of all 18221 false negatives.

Category	Share	Interpretation
Sibling (common dependency)	94.68%	Fixable: both files depend on a shared module; reachable only by a mixed-direction path
Forward-only dependency	2.13%	Fixable: excluded by design
No static path at all	3.19%	True ceiling: ground-truth limit

Table 3: Tuned 30-task set versus 80 held-out real-issue tasks, thresholds unchanged.

Metric	Tuned ($n=30$)	Held-out ($n=80$)
Gate F1	1.00	0.37
Gate precision	1.00	0.45
Gate recall	1.00	0.31
Routing exact-tier	1.00	0.53
Routing within-1-tier	1.00	0.91
Cost saving	62.1%	see Table 4

of real code; it was defeated by answering a narrower question than the one that matters.

A construction defect. Separately, and reported as a labelled diagnostic rather than substituted into the numbers above, we found a bug. The world-model’s module-qualified-name derivation collides with absolute imports in `src/-layout` packages, creating disconnected phantom nodes. A guard exists for precisely this case but never fires: its membership test is defeated by the graph library’s edge-insertion auto-vivification, which creates the node before the guard inspects it. This explains why recall was approximately zero in exactly the six `src-layout` repositories and non-negligible only in the three flat-layout ones. A naive repair of the guard raises pooled recall by 14.5 $\times$, but is unsafe (§6); the safe repair we actually apply recovers 11.0 $\times$, and that is the figure we report throughout.

5 Evaluation II: Routing and Gating

We scored the router and gate on 80 tasks drawn from real GitHub issues and pull requests, with independent gold labels for cost tier and for whether the request is under-specified enough that an agent should halt. **No threshold, weight, or pattern was modified.**

Label noise first. Agreement between two independent labelling passes was $\kappa = 0.952$ (quadratic-weighted) for tier and $\kappa = 0.516$ for the halt decision. Tier labels are near-perfect; halt labels are moderate. We report this before the accuracy numbers because it bounds how much of the degradation below can be attributed to label ambiguity rather than to the mechanism — and for the halt decision, some of it can.

Gate F1 falls from 1.00 to 0.37, driven by recall collapsing to 0.31: the gate fails to flag 11 of the 16 genuinely under-specified issues. Exact-tier routing accuracy falls to 0.53, though within-one-tier accuracy is comparatively robust at 0.91 — most routing errors are off-by-one rather than wild. Cost requires two orthogonal distinctions the tuned demonstration never had to make, because on that set almost every routed output was correct. *Framing:* whether we count only the first, initially-routed attempt, or everything the pipeline actually spent including each escalation retry. *Gating:* whether a dollar saved counts when the cheaper output is wrong. Table 4 gives all four cells.

Read in the order that matters. Routing does identify that most of these tasks do not need the premium tier, so on first attempts alone it saves 59.5% in raw dollars — close to the demonstration’s 62.1%, and the one cell where the tuned result survives. But almost

Table 4: Held-out cost saving versus an always-premium baseline, by framing and gating (95% bootstrap CIs). The tuned demonstration's 62.1% corresponds to the top-left cell: first-attempt, ungated.

	Ungated	Correctness-gated
First attempt only	59.5% [50.5, 68.2]	3.6% [0.0, 8.5]
Total pipeline spend	-20.2% [-27.5, -12.3]	2.8% [-0.8, 7.8]

none of that cheaper output is *correct*: gating the same first-attempt framing on the judge's verdict collapses the saving to 3.6%. And the pipeline does not stop at the first attempt. Counting what it actually spent chasing correctness up the ladder, the raw saving is -20.2%: the router costs *more* than always sending every task to the premium tier, because it pays for cheap, then mid, then premium in sequence and still mostly fails. The gated total-spend figure, 2.8%, has a confidence interval spanning zero.

The honest summary is not that the saving shrank but that it inverted. On this task distribution a complexity router is a net cost *increase* once its retries are counted, and the single headline number the demonstration produced (62.1%) turns out to be the most favourable of four available cells.

The gate's confidence score is non-monotonic. Binned calibration is not monotone in the score: decision accuracy is 0.93 in the middle bin ($n=28$) and 0.94 in the highest ($n=16$), but dips to 0.50 in the second-highest bin ($n=4$). We flag this as a caution rather than a finding: with four tasks in the affected bin the dip is well within sampling noise, and the top bin is in fact well calibrated. What it does establish is that the score is not reliably ordered across its range, so choosing an operating point from it requires more evidence than this sample provides.

Verification coverage. Of the 64 well-specified held-out tasks, *none* admitted execution-based verification: real issue text does not come with runnable input/output examples. Correctness was therefore judged by a model-based criterion, which is weaker, and we mark the cost figure accordingly. One candidate execution match was found by pattern search and rejected on inspection as prose describing a source-code branch rather than an invocable example.

6 Repair and Re-Evaluation

A refutation that stops at the negative number leaves the more useful question unasked: was the mechanism wrong, or merely built wrong? §4 says 96.8% of the missed impact is reachable in a graph the tool already constructs. We therefore repaired both defects and re-ran the same frozen protocol.

Honest parameter setting. The traversal repair introduces new parameters, so it cannot be evaluated on the corpus that motivated it — that is the error this paper is about. Before writing any repair code we declared a stratified split: six **tuning** repositories (rich, httpx, click, flask, jinja, attrs) and three **held-out** repositories (loguru, requests, pytest), stratified by package layout in proportion to the corpus. All eight parameters were fixed by grid

Table 5: Repair sequence under the frozen protocol. Row (d) is the headline: three repositories never seen during parameter selection. Rows (a)–(b) are pooled over all nine.

	Configuration	P	R	F1
(a)	As shipped (reproduced exactly)	0.398	0.022	0.042
(b)	+ construction fix	0.304	0.242	0.270
(c)	+ traversal fix, <i>tuning</i>	0.461	0.668	0.545
(d)	+ traversal fix, held-out	0.305	0.653	0.416
(d')	same, at tuned $t=0.10$	0.320	0.647	0.428
	grep baseline, held-out	0.269	0.601	0.371
	<i>static ceiling, held-out repos</i>	—	0.977	—

search on the tuning repositories only, written to a frozen parameter file, and never adjusted afterwards. The held-out repositories were touched exactly once, after freezing.

Defect 1: construction. We repaired the phantom-node merge — but not as the diagnostic suggested. The naive guard fix is *unsafe*: it merges a bare `import json` into an unrelated local `pkg.json` submodule, fabricating dependency edges that do not exist. We ground matching in the layout prefix instead. This yields a smaller recall improvement than the unsafe variant (11.0×), and we prefer it: a tool that invents edges to raise recall is worse than one that misses them.

Defect 2: traversal. We added terminal sibling and forward relations with per-relation weights and hop caps. The sibling relation must be capped by the shared dependency's in-degree — admitting every sibling of a high-traffic utility module implicates most of the codebase. A post-freeze sensitivity check on the tuning repositories confirms the cap does real protective work rather than passing through unbounded.

Result. We headline row (d) at the canonical $t=0.02$ used for the as-shipped baseline, so the comparison is like-for-like. On repositories never seen during parameter selection the repaired oracle reaches $F1 = 0.416$ (95% CI [0.382, 0.446]) against grep's 0.371 (CI [0.332, 0.405]). This reverses the as-shipped ordering, in which grep won by roughly an order of magnitude. Recall rises from 0.022 to 0.653, reaching 66.8% of the held-out repositories' own static ceiling (0.977), and the share of files with exactly zero recall falls from 91.0% to 19.3%. Row (d') reports the same configuration at $t=0.10$, the F1-peak of the tuning grid; it scores marginally higher, and we decline to headline it because that threshold was selected on the tuning repositories and is therefore a ninth tuned parameter rather than a pre-registered one. All 49/49 tests pass, including 13 new tests covering both fixes and the stdlib-collision safety case. Both fixes are necessary: neither alone crosses the baseline.

The win is narrow, and precision generalised worse than recall. The two confidence intervals are close, and the honest reading is a real but modest advantage, not a decisive one. More instructive is the asymmetry: recall transferred almost perfectly from tuning to held-out repositories (-0.021), while precision fell by -0.141. Some overfitting to the tuning set occurred during parameter selection — visible *only* because the split was declared in advance. Had we

tuned on all nine repositories we would have reported the better number and never known the gap existed.

7 Threats to Validity

Construct. Co-change is a proxy for semantic impact and errs in both directions. Files co-change for reasons no dependency analysis can predict — release chores, coordinated documentation, test-fixture updates — and files that ought to co-change are sometimes missed by their authors. We do not treat co-change as truth; we measure its ceiling (96.9% of ground-truth pairs are statically connected in principle) and interpret every result against it. An over-warning is likewise not necessarily an error: it may be a correct dependency that has not yet co-changed in the mined window.

Internal. The pre-registration, the tuning/held-out split, and the parameter freeze were all self-administered within one continuous working session, with no external timestamping authority: a reader must take the ordering on trust, evidenced only by the artifact store's own version timestamps and the append-only amendment trail. We consider this the central weakness of a self-evaluation and state it plainly rather than resting on the protocol's authority. The held-out task labels come from a single labelling pair; halt-decision agreement is only moderate ($\kappa = 0.52$), so part of the gate's measured degradation is label ambiguity rather than mechanism failure. Tier agreement is near-perfect ($\kappa = 0.95$), so the routing result is on firmer ground. No held-out task admitted execution-based verification, so correctness was judged by a weaker model-based criterion.

External. Nine repositories, Python only, open-source only, all library-shaped rather than application-shaped. Results may differ for applications with heavy dynamic configuration, for languages with static typing that makes dependencies more explicit, or for proprietary codebases with different commit conventions. GitHub issue text is a proxy for the prompts a developer actually gives an agent.

Conclusion. All intervals are 95% file- or task-resampled bootstrap intervals. We report them for every headline number and make no claim that survives only at a point estimate. We apply no multiple-comparison correction and claim no significance test; the comparisons are descriptive.

8 Related Work

The composition law is not ours. Read as mathematics, the product-of-independent-layer-misses form is textbook. Layer-of-protection analysis has expressed unmitigated event frequency as a product over independent protection layers since the 1990s [7]; fault-tree analysis generalises it [10]; Reason's Swiss-cheese model gives the same intuition a human-factors vocabulary [9]; and reliable-computation-from-unreliable-parts runs back to von Neumann and to N-version programming [3]. The literature also supplies the caveat any layered argument must confront: Knight and Leveson showed experimentally that independently written versions do *not* fail independently [8], so an independence assumption between layers must be defended rather than asserted. We claim no novelty for the arithmetic.

And it was independently derived, more generally, before ours. More consequentially, two concurrent preprints derive a strictly more general Bayesian form of the same law weeks before our own write-up. *Odds Law* [2] develops a decomposition algebra whose verification-odds law and k -gate geometric-amplification theorem subsume our two-layer product as a special case, adding a threshold dichotomy and an information-theoretic ceiling we do not have. Its companion *Maestro Order* [1] compiles the algebra into a controller and names the coder/test-suite pairing as its canonical software-engineering instance — the same probabilistic-generator/deterministic-checker identification we had claimed. We concede priority. What our framing retains is narrower: an argument for the split *derived* from the architecture of a frozen parametric map rather than assumed as a black-box parameter, and — the gap we can actually fill — an evaluation against real repositories, where both preprints offer only Monte Carlo simulation of an abstract solver model.

Impact analysis is forty years old and being reinvented right now. The oracle's pipeline descends directly from program slicing [11], program dependence graphs [5], and interprocedural slicing [6]. Its ground truth follows the co-change mining line [12, 13], and its confidence decay has a decade-old empirical precedent in dependency-distance studies of change propagation. Concurrently, at least four 2026 preprints wire comparable AST/graph analyses into LLM coding-agent loops — source-to-test dependency mapping, library-defect propagation, persistent codebase knowledge graphs, and assumption localisation — and, notably, *none of them claims perfect recall*. That the sub-field is simultaneously decades old and actively re-invented raises the evidential bar rather than lowering it.

Pre-registration in software engineering. Pre-registration and Registered Reports are not new to empirical software engineering, where they were introduced to address exactly the degrees of freedom we exploit here unintentionally. We claim no novelty for the practice. What we add is narrower: a concrete instance of applying it to LLM-agent mechanism evaluation, where self-built demonstrations are currently the prevailing evidence standard, together with a measurement of how large the resulting gap can be.

Routing and abstention. Cost-aware cascading was named by FrugalGPT [4] and the successor literature is large, with learned and preference-trained routers now standard. The most pointed prior work for us argues that deployed routers rely on uncalibrated confidence and per-workload threshold tuning, and replaces hand-set thresholds with calibrated error probabilities — a critique that lands squarely on our own tuned-rubric methodology, and which our held-out result independently confirms. Selective prediction with a reject option is older still, running back to Chow's rule. Surveys of LLM self-correction report increasingly negative results, which supports rather than undermines the case for an *external* deterministic check.

9 Discussion

Three lessons generalise beyond our two mechanisms.

A self-built demonstration can overstate recall by more than an order of magnitude. Our demo package was not adversarial or dishonest: ten files, five plausible mutations, honestly

Impact Oracle repair: pooled metrics at each stage's peak-F1 threshold (error bars: 95% bootstrap CI, file-resampled B=2000; diamond: grep baseline on the same repo set)

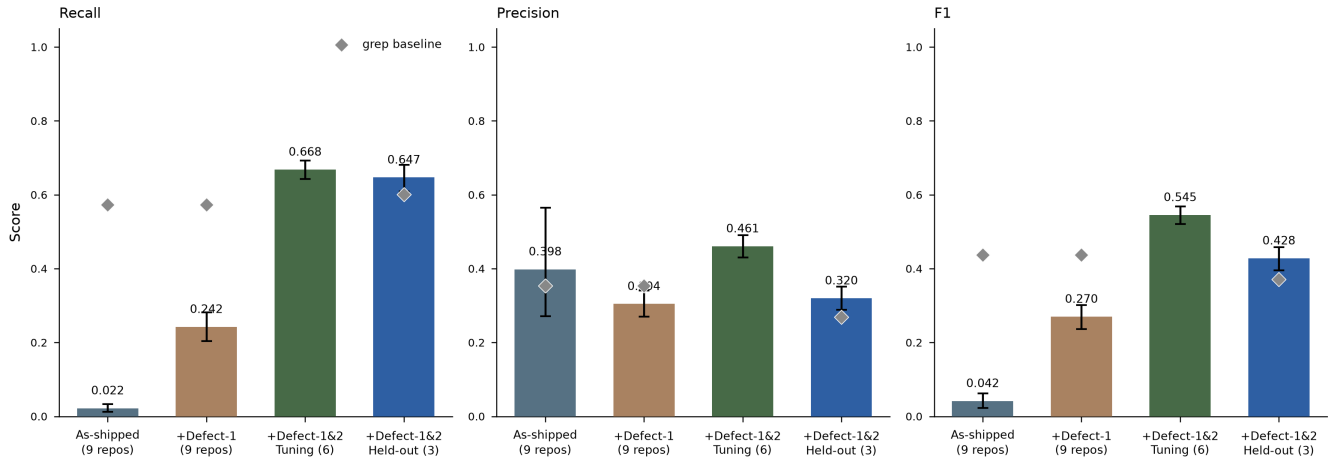


Figure 2: The repair sequence (a)→(d) with 95% bootstrap CIs, against the grep baseline and the static ceiling.

labelled as a demonstration. It still reported 1.000 where real repositories gave 0.022. The demo was unrepresentative in a specific and instructive way — it used a flat package layout, and the defect that destroyed recall only manifests under `src/` layouts. A demonstration selects its own structure, and structure was the variable that mattered.

Tuning and evaluating on the same set does not degrade gracefully. The gate did not fall from 1.00 to 0.85; it fell to 0.37, and its confidence score is non-monotonic in the region where an operating point would be chosen. The failure is qualitative, not a matter of degree.

Report the ceiling, not just the score. Decomposing false negatives structurally was what turned an uninterpretable 2.2% into an actionable finding: 96.8% fixable, 3.2% a genuine proxy limit. Without the ceiling, the number is merely discouraging; with it, it is a specification for the repair.

10 Conclusion

We pre-registered an evaluation of two LLM-agent reliability mechanisms we had built ourselves and reported strong results for, then ran it against data we did not construct. Both headline results collapsed: recall 1.000 → 0.022, gate F1 1.00 → 0.37, and a cost saving that inverts from +62.1% to −20.2% once the pipeline's escalation retries are counted. A grep baseline with no dependency graph outperformed the graph-based oracle. We diagnosed the causes structurally, repaired both defects, and re-evaluated under the same frozen protocol with a declared held-out split.

We think the protocol is the transferable contribution. The mechanisms may yet be made to work — 96.8% of the missed impact is reachable in a graph the tool already builds — but the evidence standard that told us they already worked was the thing most in need of repair.

Artifact availability

All materials are available: the pre-registration with its append-only amendment trail, the corpus manifest with pinned commit SHAs, the mined co-change ground truth, the held-out task set with its labelling protocol and both labelling passes, the tuning/held-out split declaration and frozen parameter file, the repaired package with its test suite, and the analysis code that regenerates every number and figure in this paper. References are withheld for anonymous review and will be released on acceptance.

Design provenance

The design principles behind these mechanisms — verify a report before acting on it, do not act without knowledge of consequences, treat a delegated capability as a trust — were derived from an ethical-epistemic reading documented in full in the extended version of this work. That reading motivated which safeguards were considered mandatory rather than optional; it makes no technical claim, and every result above stands or falls on the engineering evidence alone.

References

- [1] Hidayet Aksu. 2026. Maestro Order: A Model-Agnostic Orchestration Harness. arXiv preprint. doi:10.48550/arxiv.2606.23983
- [2] Hidayet Aksu. 2026. Odds Law: The Decomposition Algebra — On How Intelligence Organizes Itself to Solve Difficult Problems Reliably. arXiv preprint. doi:10.48550/arxiv.2606.15712
- [3] Algirdas Avizienis and John P. J. Kelly. 1985. The N-Version Approach to Fault-Tolerant Software. IEEE Transactions on Software Engineering. doi:10.1109/tse.1985.231893
- [4] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv preprint. doi:10.48550/arxiv.2305.05176
- [5] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. 1987. The program dependence graph and its use in optimization. ACM TOPLAS. doi:10.1145/24039.24041
- [6] Susan Horwitz, Thomas Reps, and David Binkley. 1990. Interprocedural slicing using dependence graphs. ACM TOPLAS. doi:10.1145/77606.77608

- [7] Arthur M. Dowell III. 1998. Layer of protection analysis for determining safety integrity level. ISA Transactions. doi:10.1016/s0019-0578(98)00018-4
- [8] John C. Knight and Nancy G. Leveson. 1986. An experimental evaluation of the assumption of independence in multiversion programming. IEEE Transactions on Software Engineering. doi:10.1109/tse.1986.6312924
- [9] James Reason. 1990. Human Error. Cambridge University Press. doi:10.1017/cbo9781139062367
- [10] Enno Ruijters and Mariëlle Stoelinga. 2015. Fault tree analysis: A survey of the state-of-the-art in modeling, analysis and tools. Computer Science Review.
- doi:10.1016/j.cosrev.2015.03.001
- [11] Mark Weiser. 1984. Program Slicing. IEEE Transactions on Software Engineering. doi:10.1109/tse.1984.5010248
- [12] Thomas Zimmermann, Peter Weisgerber, Stephan Diehl, and Andreas Zeller. 2004. Mining Version Histories to Guide Software Changes. Proc. ICSE. doi:10.1109/icse.2004.1317478
- [13] Thomas Zimmermann, Peter Weisgerber, Stephan Diehl, and Andreas Zeller. 2005. Mining version histories to guide software changes. IEEE Transactions on Software Engineering. doi:10.1109/tse.2005.72